

ASSIGNMENT 12.1

Algorithms with AI Assistance – Sorting, Searching, and Optimizing Algorithms

Sarika Palle

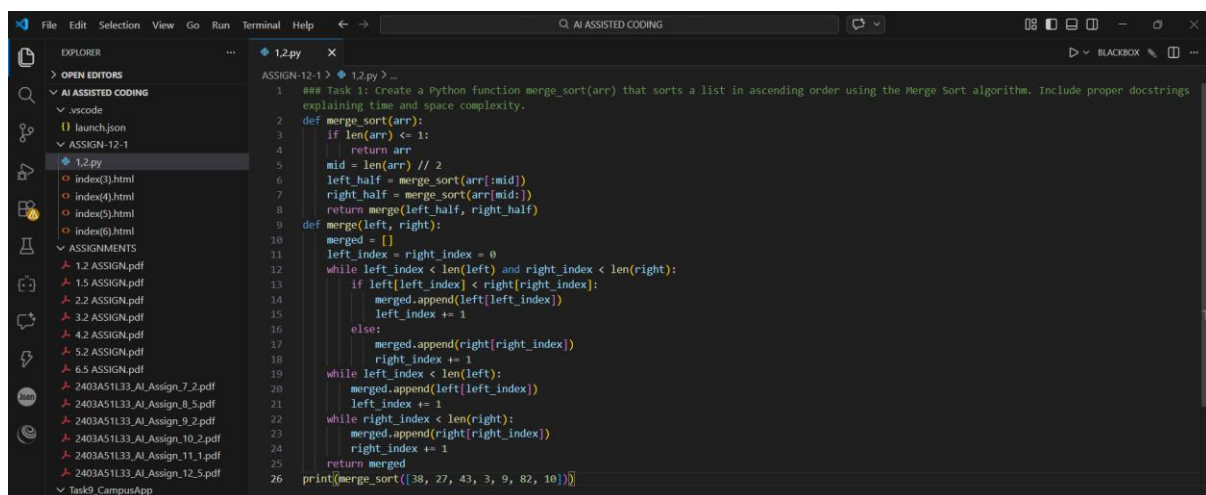
2403a51l33

B-52

Task Description #1 (Sorting – Merge Sort Implementation):

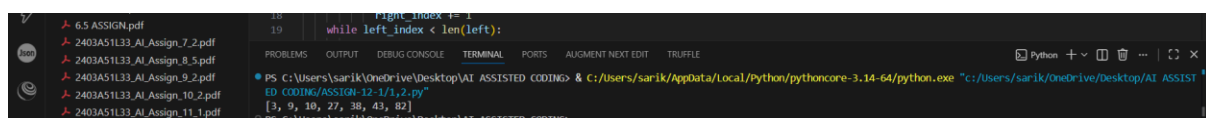
PROMPT: Create a Python function `merge_sort(arr)` that sorts a list in ascending order using the Merge Sort algorithm. Include proper docstrings explaining time and space complexity.

CODE:



```
1  """ Task 1: Create a Python function merge_sort(arr) that sorts a list in ascending order using the Merge Sort algorithm. Include proper docstrings
2  explaining time and space complexity.
3  """
4  def merge_sort(arr):
5      if len(arr) <= 1:
6          return arr
7      mid = len(arr) // 2
8      left_half = merge_sort(arr[:mid])
9      right_half = merge_sort(arr[mid:])
10     return merge(left_half, right_half)
11
12     def merge(left, right):
13         merged = []
14         left_index = right_index = 0
15         while left_index < len(left) and right_index < len(right):
16             if left[left_index] < right[right_index]:
17                 merged.append(left[left_index])
18                 left_index += 1
19             else:
20                 merged.append(right[right_index])
21                 right_index += 1
22         while left_index < len(left):
23             merged.append(left[left_index])
24             left_index += 1
25         while right_index < len(right):
26             merged.append(right[right_index])
27             right_index += 1
28         return merged
29
30     print(merge_sort([38, 27, 43, 3, 9, 82, 10]))
```

OUTPUT:



```
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING> python 12.py
[3, 9, 10, 27, 38, 43, 82]
```

EXPLANATION: AI generated a divide-and-conquer based sorting solution with detailed complexity analysis. The output was verified using multiple test cases to ensure correctness and efficiency.

Task Description #2 (Searching – Binary Search with AI Optimization):

PROMPT: Develop a Python function `binary_search(arr, target)` to search for an element in a sorted list and return its index or -1 if not found. Add documentation describing best, average, and worst-case complexities.

CODE:

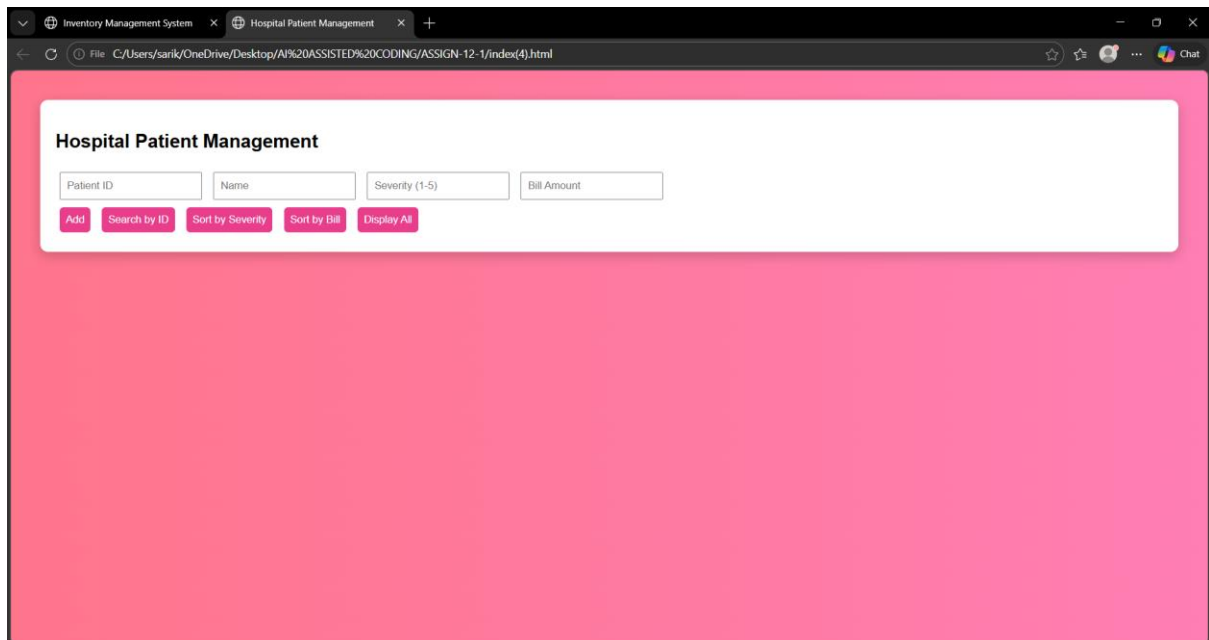
OUTPUT:

```
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING\ASSIGN-12-1> python 1.2.py
Element 5 found at index: 4
```

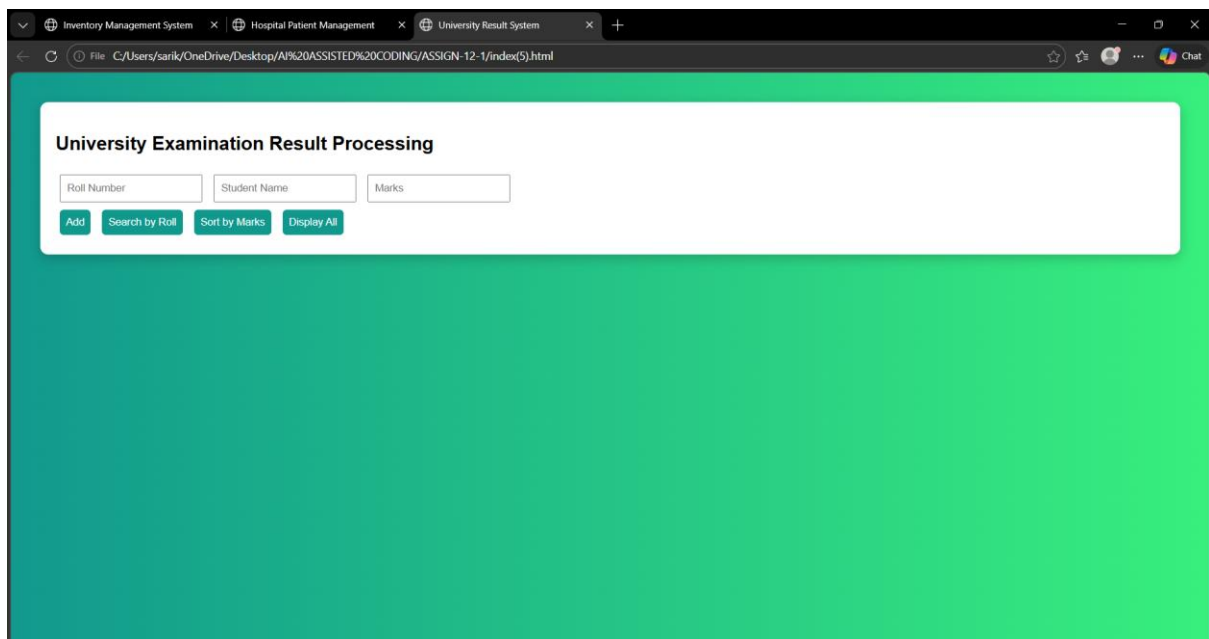
EXPLANATION: AI produced an optimized searching algorithm that reduces comparisons using a halving strategy. The function was tested with different inputs to validate accuracy and performance.

Task Description #3 (Real-Time Application – Inventory Management System):

Task description #4 (Smart Hospital Patient Management System):



Task Description #5 (University Examination Result Processing System):



Task Description #6 (Online Food Delivery Platform):

